



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Departament d'Enginyeria Minera, Industrial
i TIC

PRÀCTICA 5

Cronòmetre

Jordi Garcia

Mateu Magem



1. INTRODUCCIÓ

L'objectiu de la pràctica és la descripció i implementació d'un cronòmetre digital a la FPGA de la tarja Basys 3. Les principals característiques d'aquest es descriuen a continuació:

- Ha d'utilitzar com a font de senyal de rellotge l'oscil·lador integrat de 100 Mhz.
- El valor de comptatge serà mostrat en els 4 displays de 7 segments de la Basys III.
- Hi haurà dos formats possibles per mostrar el valor de comptatge pels displays, dels quals n'elegirem un mitjançant un dels interruptors de la tarja: o MM:SS (minuts i segons). o SS:CC (segons i centèsimes de segon).
- Disposarà d'un polsador amb la funció START/STOP.
- Un altre polsador tindrà la funció de LAP, amb la qual podrem visualitzar un temps parcial mentre internament el cronòmetre va avançant. Quan es premi aquest polsador, la visualització s'aturarà fins que el tornem a prémer, moment en el qual tornarem a visualitzar el temps actual en moviment.
- La posada a zero, coincidirà amb el polsador de RESET del sistema.
- Per últim, un polsador ens permetrà visualitzar l'últim temps parcial emmagatzemat. Es mostrarà aquest valor pels displays mentre el polsador es mantingui activat.



2. PROJECTES I ARQUITECTURES

2.1. Divisor del senyal de rellotge

Atès que la funcionalitat a implementar és la d'un cronòmetre necessitarem divisors de rellotge que divideixin el senyal de rellotge de la placa a les freqüències corresponents a cadascuna de les unitats de temps que facin falta.

Per això implementarem els següents divisors que empaquetarem com a IPs:

- Divisor de rellotge de mil·lèsimes de segon
- Divisor de rellotge de centèsimes de segon
- Divisor de rellotge de dècimes de segon
- Divisor de rellotge de segons
- Divisor de rellotge de desenes de segons
- Divisor de rellotge de minuts
- Divisor de rellotge de desenes de minuts



El cronòmetre consta d'un divisor de rellotge de partida que redueix la freqüència del rellotge de la placa Basys 3 de 100 MHz a 1 kHz. Aquest divisor emet un pols cada mil·lèsima de segon i ens serveix per refrescar els visualitzadors de 7 segments. També serveix per comptar centèsimes de segon dividint el seu senyal per 10 amb un divisor posterior.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity clk_divider is
    generic(eoc: integer:=1000000);
    Port ( clk : in STD_LOGIC;
          reset : in STD_LOGIC;
          clk_div : out STD_LOGIC);
end clk_divider;

--Divisor de Clock de "eoc" vegades
architecture Behavioral of clk_divider is
    signal div_cnt: integer range 0 to eoc;
begin
    divisor_clk: process (clk, reset) begin
        if clk'event and clk = '1' then
            clk_div <= '0';
            if reset = '1' then
                div_cnt <= 0;
            elsif div_cnt = eoc then
                div_cnt <= 0;
                clk_div <= '1';
            else div_cnt <= div_cnt + 1;
            end if;
        end if;
    end process;
end Behavioral;
```



El codi dels divisors de centèsimes de segon, dècimes de segon, segons, desenes de segons, minuts i desenes de minuts és el mateix. Cada un d'aquests subprogrames rep el senyal *Clk_E* de l'anterior divisor i compta, és a dir el divisor de dècimes de segon compta 10 flancs ascendents dels divisor de centèsimes de segon i emet un pols i així consecutivament per la resta de divisors.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_arith.ALL;

entity div_temp_1 is
    generic( n : integer:=10);
    Port ( Clk_E : in STD_LOGIC;
          reset : in STD_LOGIC;
          compt : out STD_LOGIC_VECTOR (3 downto 0);
          thick : out STD_LOGIC);
end div_temp_1;

architecture Behavioral of div_temp_1 is
    signal compta: integer range 0 to n;
begin
    comptadortemps: process(Clk_E,compta) begin
        if rising_edge(Clk_E) then
            thick <= '0';
            if reset = '1' then
                compta <= 0;
            else
                if compta = n-1 then
                    compta <= 0;
                    thick <= '1';
                else
                    compta <= compta +1;
                end if;
            end if;
        end if;
    end process;
    compt <= conv_std_logic_vector(compta,4);
end Behavioral;
```



L'únic divisor que és diferent és el divisor dels minuts que, en lloc de comptar fins a 10, compta fins a 6, ja que 6 desenes de segons són 1 minut.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_arith.ALL;

entity div_temp_5 is
    generic( n : integer:=6);
    Port ( Clck_E : in STD_LOGIC;
          reset : in STD_LOGIC;
          compt : out STD_LOGIC_VECTOR (3 downto 0);
          thick : out STD_LOGIC);
end div_temp_5;

architecture Behavioral of div_temp_5 is
    signal compta: integer range 0 to n;
begin
    comptadortemps: process(Clck_E,compta) begin
        if rising_edge(Clck_E) then
            thick <= '0';
            if reset = '1' then
                compta <= 0;
            else
                if compta = n-1 then
                    compta <= 0;
                    thick <= '1';
                else
                    compta <= compta +1;
                end if;
            end if;
        end if;
    end process;

    compt <= conv_std_logic_vector(compta,4);
end Behavioral;
```

2.2. Funció START/STOP

La funció START STOP realitza la funció d'iniciar el cronòmetre o pausar-lo en qualsevol moment del seu funcionament.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity startstop is
    Port ( clk : in STD_LOGIC;
          reset: in STD_LOGIC;
          stst : in STD_LOGIC;
          crono_on : out STD_LOGIC);
end startstop;

architecture Behavioral of startstop is
    type estat is (s1,s2);
    signal state, nxstate:estat;
begin

    sync_proc:process(clk,reset)
    begin
        if (clk'event and clk='1') then
            if reset='1' then
                state<=s1;
            else
                state<=nxstate;
            end if;
        end if;
    end process;

    next_state_decode:process(state, stst)
    begin
        nxstate<=state;
        case(state) is
            when s1=>
                if stst = '1' then
                    nxstate<=s2;
                end if;
            when s2=>
                if stst = '1' then
                    nxstate<=s1;
                end if;
            end case;
        end process;
    end
```



```
        end if;  
    end case;  
end process;  
  
output_decode:process(state)  
begin  
    if state=s1 then  
        crono_on<='0';  
    elsif state=s2 then  
        crono_on<='1';  
    end if;  
end process;  
end Behavioral;
```



2.3. Funció LAP

La funció LAP és la encarregada d'activar a '1' la sortida quan el botó de LAP es pulsi.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity LAP is
    Port ( clk : in STD_LOGIC;
          reset:in STD_LOGIC;
          lapbt : in STD_LOGIC;
          lap_on : out STD_LOGIC);
end LAP;

architecture Behavioral of LAP is
    type estat is (espera, comptador, espera_lliberacio, espera1,
    comptadors, espera_lliberacio1);
    signal next_state:estat;
    signal comptador1: integer range 0 to 1000;
    signal comptador2: integer range 0 to 1000;
    signal m: std_logic:='0';
begin

    next_state_decode:process(clk)
    begin
        if rising_edge(clk) then
            if reset = '1' then
                next_state <= espera;
                comptador1 <= 0;
                comptador2 <= 0;
            else
                case next_state is
                    when espera=>
                        if lapbt = '1' then
                            next_state<=comptador;
                        else
                            next_state<=espera;
                        end if;
                    when comptador=>
                        if comptador1=1000 then
```



```
comptador1 <= 0;
if lapbt = '1' then
    m<='1';
    next_state <= espera_lliberacio;
else
    next_state <= espera;
end if;
else
    comptador1<=comptador1+1;
end if;
when espera_lliberacio =>
    if lapbt='0' then
        next_state <= esperal;
    else
        next_state <= espera_lliberacio;
    end if;

when esperal=>
    if lapbt = '1' then
        next_state<=comptadordos;
    else
        next_state<=esperal;
    end if;
when comptadordos=>
    if comptador2=1000 then
        comptador2 <=0;
        if lapbt = '1' then
            m<='0';
            next_state <= espera_lliberacio1;
        else
            next_state <= esperal;
        end if;
    else
        comptador2<=comptador2+1;
    end if;
when espera_lliberacio1 =>
    if lapbt='0' then
        next_state <= espera;
    else
        next_state <= espera_lliberacio1;
    end if;
```



```
        when others=>
            comptador1 <= 0;
            comptador2 <= 0;
            next_state<=espera;
        end case;
    end if;
end if;
end process;
lap_on<=m;
end Behavioral;
```



2.4 Visualitzador

El visualitzador és el programa encarregat de governar la funció LAP, START/STOP i el de canvi d'unitats per tal d'enviar el valor corresponent als displays de la placa en cada cas. Per això fem servir una màquina d'estats, variables internes i un registre per tal de guardar el valor del cronòmetre en l'instant que s'activa LAP.

```
library IEEE;
use IEEE.std_logic_arith.ALL;
use IEEE.std_logic_unsigned.All;
use IEEE.STD_LOGIC_1164.ALL;

entity visualitzador is
  Port (
    btnr:in std_logic;
    reset:in std_logic;
    clk:in std_logic;
    mssc:in std_logic;
    st:in std_logic;
    lap:in std_logic;
    centiseg:in std_logic_vector(3 downto 0);
    deciseg:in std_logic_vector(3 downto 0);
    seg:in std_logic_vector(3 downto 0);
    decaseg:in std_logic_vector(3 downto 0);
    min:in std_logic_vector(3 downto 0);
    decamin:in std_logic_vector(3 downto 0);
    disp0:out std_logic_vector(3 downto 0);
    disp1:out std_logic_vector(3 downto 0);
    disp2:out std_logic_vector(3 downto 0);
    disp3:out std_logic_vector(3 downto 0);
    crono_enable:out std_logic);
end visualitzador;

architecture Behavioral of visualitzador is
  type estat is (s1,s2,s3);
  signal state, next_state:estat;

  -- Registre de volta
  type registre is record
    r1: std_logic_vector(3 downto 0);
    r2: std_logic_vector(3 downto 0);
```



```
r3: std_logic_vector(3 downto 0);
r4: std_logic_vector(3 downto 0);
r5: std_logic_vector(3 downto 0);
r6: std_logic_vector(3 downto 0);
end record;
signal reg_lap:registre;
begin

sync_proc:process(clk)
begin
    if (clk'event and clk='1') then
        if(reset='1') then
            state<=s1;
        else
            state<=next_state;
        end if;
    end if;
end process;

next_state_decode:process(state, lap,centiseg)
begin
    next_state<=state;
    case(state) is
        when s1=>
            if lap = '1' then
                next_state<=s2;
            end if;
        when s2=>
            if centiseg = 1 or centiseg= 2 or centiseg=3 or
centiseg=4 or centiseg=5 or centiseg=6 or centiseg=7 or
centiseg=8 or centiseg=9 then
                next_state<=s3;
            end if;
        when s3=>
            if lap= '0' then
                next_state<=s1;
            end if;
        end case;
    end process;
```



```
output_decode:process(state, reg_lap)
begin
    if state=s1 then
    elsif state=s2 then
        reg_lap.r1<=centiseg;
        reg_lap.r2<=deciseg;
        reg_lap.r3<=seg;
        reg_lap.r4<=decaseg;
        reg_lap.r5<= min;
        reg_lap.r6<=decamin;
    elsif state=s3 then
    end if;
end process;

-- Interruptor del rellotge principal
crono_enable<=st;

-- Canviador de segons i centèsimes a minuts i segons
visual_disp0:process(seg,centiseg, mssc, btnr, lap,reg_lap)begin
if mssc = '1' then
    if(btnr = '1' or lap= '1') then
        disp0<=reg_lap.r3;
    else
        disp0<=seg;
    end if;
else
    if (btnr = '1' or lap='1') then
        disp0<=reg_lap.r1;
    else
        disp0<=centiseg;
    end if;
end if;
end process;
visual_disp1:process(decaseg,deciseg, mssc, btnr,
lap,reg_lap)begin
if mssc = '1' then
    if(btnr = '1' or lap= '1') then
        disp1<=reg_lap.r4;
    else
        disp1<=decaseg;
    end if;
end if;
```



```
else
    if (btnr = '1' or lap='1') then
        disp1<=reg_lap.r2;
    else
        disp1<=deciseg;
    end if;
end if;
end process;
visual_disp2:process(min,seg, mssc, btnr, lap,reg_lap)begin
if mssc = '1' then
    if(btnr = '1' or lap= '1') then
        disp2<=reg_lap.r5;
    else
        disp2<=min;
    end if;
else
    if (btnr = '1' or lap='1') then
        disp2<=reg_lap.r3;
    else
        disp2<=seg;
    end if;
end if;
end process;
visual_disp3:process(decamin,decaseg, mssc, btnr,
lap,reg_lap)begin
if mssc = '1' then
    if(btnr = '1' or lap= '1') then
        disp3<=reg_lap.r6;
    else
        disp3<=decamin;
    end if;
else
    if (btnr = '1' or lap='1') then
        disp3<=reg_lap.r4;
    else
        disp3<=decaseg;
    end if;
end if;
end process;
end Behavioral;
```

2.5. Descodificador dels visualitzadors de 7 segments

El descodificador de 7 segments és el subprograma encarregat mostrar a cada visualitzador de 7 segments el valor pertinent a la freqüència adequada. Per realitzar això, utilitza el divisor de rellotge de mil·lèsimes de segon refrescant els visualitzadors a un 1kHz (canvia l'ànode i, per tant, el visualitzador actiu cada 1ms) i descodificant els vectors de bits de entrada a la corresponent configuració dels càtodes.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity seg7_coder is
    Port ( char0, char1, char2, char3 : in STD_LOGIC_VECTOR (3
downto 0);
        clk : in STD_LOGIC;
        reset : in STD_LOGIC;
        clk_div : in STD_LOGIC;
        cat : out STD_LOGIC_VECTOR (7 downto 0);
        an : out STD_LOGIC_VECTOR (3 downto 0));
end seg7_coder;

architecture Behavioral of seg7_coder is
    signal digit_act: integer range 0 to 3;
    signal sg: std_logic_vector (3 downto 0);

begin
    -- 2 Comptador que assigna la entrada que toca
    digit_actual: process(clk, reset) begin
        if clk'event and clk = '1' then
            if reset = '1' then
                digit_act <= 0;
            elsif clk_div = '1' then
                if digit_act = 3 then
                    digit_act <= 0;
                else digit_act <= digit_act + 1;
                end if;
            end if;
        end if;
    end if;
```



```
end process;

-- 3 Valors dels catodes
selector: process(digit_act, sg, char0, char1, char2, char3)
begin
    -- Assignar conjunt de switches a sg
    case digit_act is
        when 0 =>
            sg <= char0;
        when 1 =>
            sg <= char1;
        when 2 =>
            sg <= char2;
        when 3 =>
            sg <= char3;
    end case;
end process;

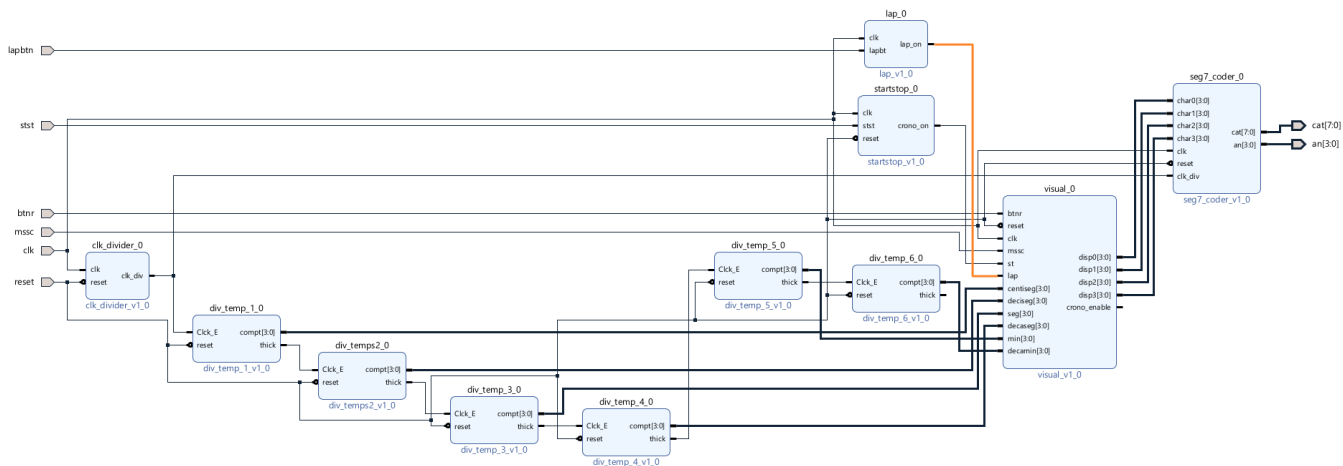
with sg select cat <=
    "00000011" when "0000", --0
    "10011111" when "0001", --1
    "00100101" when "0010", --2
    "00001101" when "0011", --3
    "10011001" when "0100", --4
    "01001001" when "0101", --5
    "01000001" when "0110", --6
    "00011111" when "0111", --7
    "00000001" when "1000", --8
    "00011001" when "1001", --9s
    "11111101" when "1111", -- -
    "11111111" when others;

with digit_act select an <=
    "1110" when 0,
    "1101" when 1,
    "1011" when 2,
    "0111" when 3;

end Behavioral
```

3. INTEGRACIÓ

El resultat de la integració de tots els subprogrames descrits anteriorment resulta en el següent diagrama de blocs:





4. CONCLUSIONS

En aquesta pràctica hem implementat la lògica interna i la visualització d'un cronòmetre amb capacitat per iniciar-se, aturar-se, reiniciar-se i emmagatzemar un valor en un registre asíncronament, a més a més de poder-se elegir el mode visualització entre segons i centesimes de segon i minuts i segons.